CMake Mastery

From Single Binary to Real-World Build Graphs

Python/*C++* 개발자를위한실전빌드시스템책

단일실행파일, 라이브러리분리, 외부패키지, 테스트, 설치,
Presets, Toolchain, LLVM/MLIR 연동까지한흐름으로정리한학습서

2026

Contents

1 왜 CMake 를마스터해야하는가 **1**
1.1 CMake 가실제로하는일 1

2 가장작은 CMake 프로젝트 **3**
2.1 최소구성 3
2.2 자주쓰는기본설정 3
2.3 빌드디렉토리를분리하는이유 4

3 타겟중심사고 **5**
3.1 파일이아니라타겟을본다 5
3.2 PUBLIC, PRIVATE, INTERFACE 5
3.3 header-only 라이브러리 6

4 프로젝트구조를어떻게쪼개는가 **7**
4.1 권장디렉토리구조 7
4.2 루트와서브디렉토리책임분리 7
4.3 언제 add_subdirectory 를쓰나 8

5 컴파일옵션과정의를다루는법 **9**
5.1 전역옵션보다타겟옵션 9
5.2 빌드타입과 generator expression 9
5.3 경고플래그모듈화 10

6 링크, 라이브러리, 그리고왜자꾸 undefined reference 가나는가 **11**
6.1 컴파일과링크는다른단계다 11
6.2 정적라이브러리와공유라이브러리 11
6.3 의존성전파예시 11

7 외부패키지를찾고연결하기 **13**
7.1 find_package 의두세계 13
7.2 Imported target 을써야하는이유 13
7.3 패키지를못찾을때보는것 13

8 테스트, install, package 로이어지는흐름 **15**
8.1 CTest 연결 15

8.2 install 규칙 15
8.3 재사용가능한패키지로내보내기 15

9 configure 단계에서파일을생성하는법 17
9.1 configure_file 17
9.2 file(GENERATE) 와의차이 17

10 Presets 와 Toolchain 으로팀개발수준을올리기 19
10.1 CMakePresets.json 19
10.2 Toolchain 파일 19

11 LLVM/MLIR 프로젝트를읽는눈 21
11.1 왜별도장이필요한가 21
11.2 핵심패턴 21
11.3 라이브러리계층을읽는법 22
11.4 빈출문제 22

12 디버깅과안티패턴 23
12.1 문제가생겼을때순서 23
12.2 유용한명령 23
12.3 피해야할안티패턴 23

13 당장실전에적용할로드맵 25
13.1 1 단계: 작은 CLI 프로젝트 25
13.2 2 단계: 라이브러리분리 25
13.3 3 단계: 외부패키지와설치 25
13.4 4 단계: 대형생태계읽기 25

14 Final Quiz 27

A 암기보다중요한체크포인트 29

B 빠른명령치트시트 31

Chapter 1

왜 CMake 를마스터해야하는가

당신의목표는단순히 `cmake -B build -S .` 를외우는것이아닙니다. 프로젝트구조를설계하고, 타겟단위로의존성을관리하고, **LLVM/MLIR** 같은큰생태계와연결해도버티는빌드감각을갖추는것이핵심입니다.

이책의목표

1. 작은 C++ 프로젝트를스스로빌드할수있다.
2. 헤더경로, 컴파일옵션, 링크의존성을타겟기준으로설계할수있다.
3. 외부패키지와테스트, 설치, 재사용가능한패키지구성을이해한다.
4. LLVM/MLIR 처럼크고복잡한생태계에서 CMake 파일을읽고고칠수있다.

Python 개발자관점

Python 에서익숙한것은 `pip install`, `pyproject.toml`, 가상환경, 테스트러너입니다. C++ 에서는그역할이여러층으로나뉩니다. 컴파일러, 링커, 빌드디렉토리, 패키지설정, 설치경로를명시적으로다뤄야합니다. CMake 는이복잡성을정리해주는메타빌드시스템입니다.

1.1 CMake 가실제로하는일

CMake 는컴파일러가아닙니다. `ninja`, `make`, Xcode, Visual Studio 같은 실제빌드도구용설정파일을생성합니다. 즉, CMake 의핵심일은 "프로젝트의빌드그래프를선언" 하는것입니다.

기본흐름

```
cmake -S . -B build          # configure: 빌드 그래프 생성
cmake --build build          # build: 실제 컴파일/링크
ctest --test-dir build       # test: 테스트 실행
cmake --install build        # install: 산출물 배치
```

배운것정리

- configure 단계와 build 단계는다르다.
- CMake 를읽을때는 "무슨파일을컴파일하나" 보다 "무슨타겟을정의하나" 를먼저 봐야한다.
- 프로젝트가커질수록파일보다 타겟간관계를이해하는것이더중요하다.

Quiz

Q1. CMake 가직접하는일과하지않는일을구분해보자.

1. g++ 실행자체
2. 타겟사이의존성그래프모델링
3. Ninja 파일생성
4. 소스코드를최적화해서더빠르게만드는것

정답: 2, 3 만 CMake 의역할. 1 은생성된빌드시스템이수행하고, 4 는컴파일러/링커의역할이다.

Chapter 2

가장작은 CMake 프로젝트

2.1 최소구성

CMakeLists.txt 최소버전

```
cmake_minimum_required(VERSION 3.20)
project(kv_cli LANGUAGES CXX)

add_executable(kv
  src/main.cpp
)
```

src/main.cpp

```
#include <iostream>

int main() {
  std::cout << "kv-cli" << std::endl;
  return 0;
}
```

2.2 자주쓰는기본설정

```
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
```

의미

- `CMAKE_CXX_STANDARD 20`: C++20 사용의도
- `REQUIRED ON`: 지원안하면에러

- `EXTENSIONS OFF`: `gnu++20` 대신 `c++20` 선호

주의

실전에서는전역변수보다 `target_compile_features()` 가더안전하다. 전역설정은서브프로젝트와섞일때충돌이생기기쉽다.

2.3 빌드디렉토리를분리하는이유

```
cmake -S . -B build/debug -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake -S . -B build/release -G Ninja -DCMAKE_BUILD_TYPE=Release
```

소스디렉토리와빌드디렉토리를분리하면, 설정조합을여러개유지할수있고산출물이저장소를오염시키지않습니다.

Quiz

Q2. 아래중권장되는흐름은?

A. 프로젝트루트에서 `make`

B. 소스루트에 `CMakeCache.txt` 와객체파일을섞어서생성

C. 별도 `build/` 디렉토리에서 configure/build/test/install 수행

정답: C

Chapter 3

타겟중심사고

3.1 파일이아니라타겟을본다

CMake 초보는보통 "이파일을어떻게추가하지?" 를먼저생각합니다. 그러나유지보수가능한 CMake 는 "이타겟은어떤 API 를노출하고, 무엇에링크되고, 어떤 include path 와 compile definition 을소비자에게전파하는가?" 로설계합니다.

좋은방향의 CMake

```
add_library(kv_core
  src/store.cpp
  src/command.cpp
)

target_include_directories(kv_core
  PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_features(kv_core PUBLIC cxx_std_20)

add_executable(kv_cli
  src/main.cpp
)

target_link_libraries(kv_cli PRIVATE kv_core)
```

3.2 PUBLIC, PRIVATE, INTERFACE

키워드	의미
PRIVATE	현재타겟에서만사용. 소비자에게전파되지않음
PUBLIC	현재타겟도사용하고소비자에게도전파
INTERFACE	현재타겟자체는안쓰고소비자에게만전파

전파예시

```
target_include_directories(kv_core
  PUBLIC include
  PRIVATE src
)

target_link_libraries(kv_cli
  PRIVATE kv_core
)
```

위예시에서 `kv_cli` 는 `kv_core` 를링크하면서 `include` 경로는자동으로받지만, `src` 경로는받지않습니다.

Python 개발자관점

이개념은 Python 패키지의공개 API 와내부구현경계를나누는것과비슷합니다. `PUBLIC` 은 라이브러리사용자에게노출되는계약이고, `PRIVATE` 은내부구현세부사항입니다.

3.3 header-only 라이브러리

```
add_library(kv_flags INTERFACE)
target_compile_definitions(kv_flags INTERFACE KV_ENABLE_LOG=1)
target_include_directories(kv_flags INTERFACE include)
```

`INTERFACE` 라이브러리는소스파일이없고, "사용방법" 만배포합니다. 컴파일옵션, include 디렉토리, 컴파일정의를묶는데유용합니다.

Quiz

Q3. `kv_core` 의헤더에서 `fmt::format` 이반환타입과인자타입에등장한다. `fmt` 는 `PUBLIC`, `PRIVATE`, `INTERFACE` 중무엇이어야할까?
정답: 보통 `PUBLIC`. 헤더를소비하는쪽도그의존성을알아야하기때문이다.

Chapter 4

프로젝트구조를어떻게쪼개는가

4.1 권장디렉토리구조

```
project-root/
  CMakeLists.txt
  cmake/
    warnings.cmake
    sanitizers.cmake
  include/
    kv/store.h
  src/
    store.cpp
    main.cpp
  tests/
    store_test.cpp
  third_party/
```

4.2 루트와서브디렉토리책임분리

루트 CMakeLists.txt

```cmake
cmake_minimum_required(VERSION 3.20)
project(kv_cli VERSION 0.1.0 LANGUAGES CXX)

option(KV_BUILD_TESTS "Build tests" ON)

add_subdirectory(src)

if(KV_BUILD_TESTS)
  enable_testing()
  add_subdirectory(tests)
endif()
```

src/CMakeLists.txt

```
add_library(kv_core
  store.cpp
)

target_include_directories(kv_core
  PUBLIC ${PROJECT_SOURCE_DIR}/include
)

add_executable(kv_cli main.cpp)
target_link_libraries(kv_cli PRIVATE kv_core)
```

주의

CMAKE_SOURCE_DIR 와 PROJECT_SOURCE_DIR 를 무분별하게 섞지 마십시오. 서브프로젝트로 포함될 가능성이 있으면 CMAKE_CURRENT_SOURCE_DIR, PROJECT_SOURCE_DIR, PROJECT_BINARY_DIR 처럼 문맥에 맞는 변수를 써야 합니다.

4.3 언제 add_subdirectory 를 쓰나

- 우리 프로젝트의 일부인 내부 모듈: add_subdirectory
- 설치된 외부 패키지 찾기: find_package
- Git submodule 이나 번들된 외부 라이브러리 직접 포함: 경우에 따라 add_subdirectory, 단 정책 충돌 주의

Quiz

Q4. 루트에서 include_directories() 를 한 번 호출해서 모든 하위 타겟에 전역으로 뿌리는 방식이 왜 위험한가?
정답: 타겟 경계가 사라져서 어떤 헤더가 어디서 오는지 추적이 어려워지고, 외부 서브프로젝트와 충돌하기 쉽다. 타겟 단위 전파가 더 읽기 쉽고 재사용 가능하다.

Chapter 5

컴파일옵션과정의를다루는법

5.1 전역옵션보다타겟옵션

권장패턴

```
target_compile_features(kv_core PUBLIC cxx_std_20)

target_compile_options(kv_core PRIVATE
  -Wall
  -Wextra
  -Wpedantic
)

target_compile_definitions(kv_core PRIVATE
  KV_DATA_DIR="${CMAKE_CURRENT_SOURCE_DIR}/data"
)
```

5.2 빌드타입과 generator expression

```
target_compile_definitions(kv_core PRIVATE
  $<$<CONFIG:Debug>:KV_ENABLE_ASSERTS=1>
  $<$<CONFIG:Release>:KV_FAST_PATH=1>
)
```

`$<...>` 는 generator expression 입니다. configure 시점이아니라 generate/build 시점에해석됩니다.

Multi-config vs Single-config

- Ninja, Makefiles: 보통 single-config. `CMAKE_BUILD_TYPE` 사용.
- Visual Studio, Xcode, Ninja Multi-Config: 여러구성을동시에가짐.
- 그래서실전에서는 `if(CMAKE_BUILD_TYPE STREQUAL ...)` 보다 generator expres-

sion 이더이식성있다.

5.3 경고플래그모듈화

cmake/warnings.cmake

```
add_library(project_warnings INTERFACE)

target_compile_options(project_warnings INTERFACE
  $<$<CXX_COMPILER_ID:Clang,AppleClang,GNU>:-Wall;-Wextra;-Wpedantic>
  $<$<CXX_COMPILER_ID:MSVC>:/W4>
)
```

```
target_link_libraries(kv_core PRIVATE project_warnings)
```

Quiz

Q5. `add_compile_options(-Wall)` 와 `target_compile_options(kv_core PRIVATE -Wall)` 중후자가더나은이유는?
정답: 어떤타겟이어떤규칙을가지는지국소적으로드러나고, 외부서브프로젝트나특정타겟에원치않는옵션전파를막을수있기때문이다.

Chapter 6

링크, 라이브러리, 그리고 왜 자꾸 undefined reference 가나는가

6.1 컴파일과 링크는 다른 단계다

- 컴파일: 각 `.cpp` 를 객체파일로 변환
- 링크: 객체파일과 라이브러리를 합쳐 최종 실행파일 생성
- `undefined reference`: 헤더는 보이는데 구현 심볼이 링크에 안 들어왔다는 뜻

6.2 정적 라이브러리와 공유 라이브러리

```
add_library(kv_core STATIC
  src/store.cpp
)

add_library(kv_plugin SHARED
  src/plugin.cpp
)
```

실전 감각

- 앱 내부 모듈은 대부분 `STATIC` 또는 기본 `add_library` 로 충분하다.
- 플러그인 시스템, ABI 노출, 동적 교체가 필요할 때만 `SHARED` 를 신중하게 검토한다.
- 초반에는 "링크 그래프를 명확하게" 만드는 것이 동적 로딩보다 더 중요하다.

6.3 의존성 전파 예시

```
add_library(kv_storage src/store.cpp)
add_library(kv_commands src/command.cpp)

target_link_libraries(kv_commands PUBLIC kv_storage)
```

```

add_executable(kv_cli src/main.cpp)
target_link_libraries(kv_cli PRIVATE kv_commands)
```

위경우 `kv_cli` 는 `kv_storage` 를직접쓰지않아도, `kv_commands` 의 `PUBLIC` 링크전파덕분에함께링크됩니다.

주의

링크가우연히통과한다고해서설계가좋은것은아닙니다. 헤더에노출된의존성은 `PUBLIC`, 구현에만필요한의존성은 `PRIVATE` 로정리해야나중에리팩터링이쉬워집니다.

Quiz

Q6. `undefined reference to KVStore::set(...)` 에러가났다. 가능한원인을두가지써보라.
정답예시: `store.cpp` 가타겟소스목록에서빠졌거나, `kv_core` 라이브러리를최종실행파일에링크하지않았다.

Chapter 7

외부패키지를찾고연결하기

7.1 find_package 의두세계

1. **Module mode**: CMake 가제공하는 `FindXXX.cmake` 를찾는다.
2. **Config mode**: 패키지가설치한 `XXXConfig.cmake` 를찾는다.

```
find_package(fmt REQUIRED CONFIG)
find_package(spdlog REQUIRED CONFIG)
```

현대 CMake 에서는가능하면 Config mode 와 imported target 을쓰는것이좋습니다.

7.2 Imported target 을써야하는이유

```
target_link_libraries(kv_core
  PUBLIC
    fmt::fmt
    spdlog::spdlog
)
```

이방식은 include path, compile definition, 필요한링크플래그를패키지작성자가한번묶어서제공합니다.

주의

`include_directories(/usr/local/include)` 와 `link_directories(/usr/local/lib)` 를직접쓰는방식은현대 CMake 에서거의항상열세다. 패키지가제공하는 imported target 이있으면그걸우선사용하라.

7.3 패키지를못찾을때보는것

- `CMAKE_PREFIX_PATH`
- 패키지별 `<Pkg>_DIR`
- 설치위치안의 `lib/cmake/<Pkg>` 또는유사경로

- 아키텍처불일치여부

예시

```
cmake -S . -B build \
  -DCMAKE_PREFIX_PATH=/opt/homebrew \
  -Dfmt_DIR=/opt/homebrew/lib/cmake/fmt
```

Quiz

Q7. `find_package(MLIR REQUIRED CONFIG)` 가실패했다. 가장먼저확인할것은?
정답: MLIR 설치경로아래의 `MLIRConfig.cmake` 위치와, 그위치를가리키는 `MLIR_DIR` 또는 `CMAKE_PREFIX_PATH` 설정.

Chapter 8

테스트, install, package 로이어지는흐름

8.1 CTest 연결

```
include(CTest)

if(BUILD_TESTING)
  add_executable(store_test tests/store_test.cpp)
  target_link_libraries(store_test PRIVATE kv_core)
  add_test(NAME store_test COMMAND store_test)
endif()
```

```
ctest --test-dir build --output-on-failure
```

8.2 install 규칙

```
install(TARGETS kv_core kv_cli
  EXPORT kvTargets
  RUNTIME DESTINATION bin
  LIBRARY DESTINATION lib
  ARCHIVE DESTINATION lib
)

install(DIRECTORY include/ DESTINATION include)
```

8.3 재사용가능한패키지로내보내기

```
install(EXPORT kvTargets
  NAMESPACE kv::
  DESTINATION lib/cmake/kv
```

```
)
```

이단계까지가면다른프로젝트에서 `find_package(kv CONFIG REQUIRED)` 후 `kv::kv_core` 같은 imported target 으로소비할수있습니다.

Python 개발자관점

이감각은 Python 패키지를 wheel 로배포해서다른프로젝트가 import 하는흐름과닮았습니다. 차이는 C++ 은 ABI, 헤더, 링크옵션까지함께전달해야한다는점입니다.

Quiz

Q8. 왜 `install()` 없이도로컬빌드는되지만, 다른프로젝트에서재사용하기는어려울까?
정답: 설치규칙이없으면헤더, 라이브러리, export target 메타데이터를일관된위치와형식으로제공하지못해외부프로젝트가안정적으로소비할수없다.

Chapter 9

configure 단계에서파일을생성하는법

9.1 configure_file

```
configure_file(
  ${CMAKE_CURRENT_SOURCE_DIR}/config.h.in
  ${CMAKE_CURRENT_BINARY_DIR}/generated/config.h
  @ONLY
)
```

config.h.in

```
#pragma once

#define KV_VERSION "@PROJECT_VERSION@"
#define KV_ENABLE_CACHE @KV_ENABLE_CACHE@
```

`configure_file()` 는 configure 시점치환입니다. 빌드할때마다실행되는스크립트가아니라, 설정이바뀔때다시생성되는파일입니다.

9.2 file(GENERATE) 와의차이

- `configure_file`: configure 시점치환
- `file(GENERATE)`: generate 시점, configuration 별파일생성에유리
- `add_custom_command`: build 시점에실제작업수행

주의

configure 단계에서해야할일과 build 단계에서해야할일을혼동하면, 소스를바꿨는데파일이안갱신되거나매번불필요하게재생성되는문제가생긴다.

Quiz

Q9. protobuf 컴파일, TableGen 생성, codegen 스크립트실행은보통 `configure_file()` 보다무엇에가깝나?
정답: `add_custom_command()` 혹은해당생태계가제공하는전용매크로. 실제빌드산출물생성단계이기때문이다.

Chapter 10

Presets 와 Toolchain 으로팀개발수준을올리기

10.1 CMakePresets.json

개념예시, 실제파일은 JSON

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "dev",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/dev",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "KV_BUILD_TESTS": "ON"
      }
    }
  ]
}
```

Presets 는팀전체가같은 configure 인자를재현하도록도와줍니다. README 에긴명령을적는대신, `cmake --preset dev` 로고정할수있습니다.

10.2 Toolchain 파일

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_C_COMPILER clang)
set(CMAKE_CXX_COMPILER clang++)
```

Toolchain 파일은 cross-compiling, 특정 SDK, sysroot, 패키지매니저통합에유용합니다.

실전체크리스트

- 팀에빌드명령이길어지면 Presets 를도입한다.
- 플랫폼이나아키텍처차이를숨겨야하면 Toolchain 파일을고려한다.
- 문서보다재현가능한설정파일을우선한다.

Quiz

Q10. 개인환경별로 `CMAKE_PREFIX_PATH` 가다르고, 팀원이매번 README 의긴명령을복붙하고있다. 무엇을도입하면좋은가?
정답: 기본적으로 `CMakePresets.json`. 플랫폼/컴파일러자체까지고정해야하면필요에따라 toolchain 파일도함께사용.

Chapter 11

LLVM/MLIR 프로젝트를읽는눈

11.1 왜별도장이필요한가

당신은단순한 toy project 보다한단계더가려하고있습니다. LLVM/MLIR 기반프로젝트는 CMake 파일이길고매크로가많으며, 링크의존성이빽빽합니다. 여기서중요한것은문법암기보다 패턴인식입니다.

11.2 핵심패턴

전형적인 MLIR 설정

```
find_package(MLIR REQUIRED CONFIG)
find_package(LLVM REQUIRED CONFIG)

list(APPEND CMAKE_MODULE_PATH "${MLIR_CMAKE_DIR}")
list(APPEND CMAKE_MODULE_PATH "${LLVM_CMAKE_DIR}")

include(TableGen)
include(AddLLVM)
include(AddMLIR)
```

```
if(NOT LLVM_ENABLE_RTTI)
  add_compile_options(-fno-rtti)
endif()
```

여기서읽어야할포인트

1. 패키지설정을어디서찾는가
2. 생태계전용매크로를어떻게로드하는가
3. 프로젝트컴파일옵션이상위생태계와어떤제약을공유하는가
4. 어떤라이브러리가어떤 dialect/pass/transform 계층을담당하는가

11.3 라이브러리계층을읽는법

```
add_library(GaweeDialect
  lib/Gawee/GaweeDialect.cpp
)

target_link_libraries(GaweeDialect
  PUBLIC
    MLIRIR
    MLIRSupport
)

add_library(GaweeConversion
  lib/Conversion/GaweeToLinalg.cpp
)

target_link_libraries(GaweeConversion
  PUBLIC
    GaweeDialect
  PRIVATE
    MLIRLinalgDialect
    MLIRTransforms
)
```

여기서 `GaweeConversion` 의공개 API 가 `GaweeDialect` 타입을노출한다면 `PUBLIC` 이맞고, 내부 lowering 구현만 `MLIRLinalgDialect` 를쓰면 `PRIVATE` 가맞습니다.

11.4 빈출문제

증상	보통원인
`typeinfo for mlir::Pass` 관련링크에러	RTTI 설정불일치
`Could not find MLIRConfig.cmake`	`MLIR_DIR` 또는 prefix 경로미설정
`file not found` 헤더에러	include 경로또는 generated header 경로누락
`undefined reference` 패스생성함수	관련라이브러리미링크

Quiz

Q11. LLVM 이 RTTI 없이빌드되었는데우리프로젝트는 RTTI 를켠상태다. 가장가능성높은결과는?
정답: 컴파일은되더라도링크단계에서 `typeinfo` 관련에러가날수있다. 상위생태계와설정을맞춰야한다.

Chapter 12

디버깅과안티패턴

12.1 문제가생겼을때순서

1. configure 에러인가, build 에러인가, test/install 에러인가구분한다.
2. 문제가난타겟이무엇인지본다.
3. 그타겟의소스, include path, compile definition, link dependency 를확인한다.
4. 전역변수와매크로가흐름을숨기고있지않은지본다.

12.2 유용한명령

```
cmake -S . -B build --trace-expand
cmake --build build --verbose
cmake --graphviz=graph.dot -S . -B build
ctest --test-dir build --output-on-failure
```

12.3 피해야할안티패턴

- 루트에서 `include_directories()`, `link_libraries()` 를전역살포
- 패키지대신절대경로하드코딩
- `file(GLOB ...)` 로소스를자동수집해서변경추적불투명하게만들기
- 옵션과매크로를함수처럼섞어타겟경계를흐리기
- "일단돌아가니까" PUBLIC/PRIVATE 를대충두기

주의

`file(GLOB)` 는데모에는편하지만, 팀프로젝트에서는새파일추가가 configure 재실행과엮여예측가능성을떨어뜨린다. 명시적소스목록이더유지보수에강하다.

Quiz

Q12. CMake 디버깅에서가장먼저해야할질문은 “어느파일에서문제가났나?” 가아니라무엇인가?
정답: “어느타겟의어떤속성에서문제가났나?” 파일보다타겟이빌드그래프의기본단위이기때문이다.

Chapter 13

당장실전에적용할로드맵

13.1 1 단계: 작은 CLI 프로젝트

- `add_executable` 하나로시작
- `build/` 분리
- `target_compile_features`, `target_include_directories`, `target_link_libraries` 를익힌다

13.2 2 단계: 라이브러리분리

- `kv_core`, `kv_cli` 로분리
- PUBLIC/PRIVATE 를의식적으로설정
- 테스트타겟추가

13.3 3 단계: 외부패키지와설치

- `find_package` 와 imported target 사용
- `install()` 과 export target 구성
- Presets 로팀환경고정

13.4 4 단계: 대형생태계읽기

- LLVM/MLIR CMake 를읽을때타겟그래프를먼저본다
- 생태계전용매크로가실제로어떤타겟을만드는지추적한다
- 에러를보면 "헤더경로", "심볼정의", "상위설정불일치" 중무엇인지분류한다

배운것정리

당신의실제목표는 "CMake 문법을많이아는사람" 이아니라, 복잡한 C++ 프로젝트를겁먹지않고구조화하고디버깅하는사람이되는것입니다. mini-kvstore 같은작은 CLI 에서시작해, 라이브러리분리, 테스트, 패키지, LLVM/MLIR 연동까지자연스럽게올라가면됩니다.

Chapter 14

Final Quiz

종합문제 1

다음상황을읽고올바른설계를설명하라.

- `kv_core` 라이브러리가공개헤더에서 `fmt::format` 결과를반환한다.
- `kv_cli` 실행파일이 `kv_core` 를사용한다.
- 테스트타겟 `store_test` 도 `kv_core` 를사용한다.

해설: `kv_core` 는 `fmt::fmt` 를 `PUBLIC` 으로링크해야할가능성이높다. `kv_cli` 와 `store_test` 는 `kv_core` 만링크하면전파받을수있다.

종합문제 2

다음중대형프로젝트에서가장먼저리팩터링해야할냄새는?

A. 타겟별 `target_link_libraries` 사용

B. 전역 `include_directories()` 남발

C. `find_package(fmt CONFIG REQUIRED)` 사용

D. `install(EXPORT ...)` 제공

정답: B

종합문제 3

`gawee-opt` 링크중 `undefined reference to mlir::linalg::GenericOp::create` 가난다. 가장그럴듯한원인은?

정답: 관련 dialect/transform 라이브러리가 `target_link_libraries` 에빠졌을가능성이높다. 헤더 include 만으로는링크심볼이해결되지않는다.

Appendix A

암기보다중요한체크포인트

실전체크리스트

1. 모든것은타겟중심으로읽고쓴다.
2. 전역설정보다타겟속성을우선한다.
3. 패키지는경로하드코딩보다 imported target 을우선한다.
4. configure/build/install/test 의단계를섞지않는다.
5. 대형프로젝트에서는상위생태계의제약을먼저확인한다.

Appendix B

빠른명령치트시트

```
# Configure
cmake -S . -B build/dev -G Ninja -DCMAKE_BUILD_TYPE=Debug

# Build
cmake --build build/dev

# Run tests
ctest --test-dir build/dev --output-on-failure

# Install
cmake --install build/dev --prefix ./stage

# Trace and debug
cmake -S . -B build/dev --trace-expand
cmake --build build/dev --verbose
```